

Multi-Kernel Generalisations of Operator-Adaptive PLS for Near-Infrared Spectroscopy: mkR, MKM, and BLUP

Status: draft v0.2 (with iter8 final benchmark, 2026-05-01). All numbers in this manuscript are reproducible from `benchmark_runs/iter8_full54_champions/results.csv` and earlier benchmarks under `bench/AOM_v0/Multi-kernel/benchmark_runs/`.

This is a sister manuscript to the AOM-PLS paper at `publication/manuscript/main.tex`. It introduces three multi-kernel extensions of the AOM operator family — mkR, MKM, BLUP — sharing a common centred + trace-normalised kernel API.

Abstract

We extend the operator-adaptive partial-least-squares (AOM-PLS) framework with three multi-kernel siblings:

- **mkR** — multi-kernel Ridge with explicit per-block weights η_b , learnable from kernel-target alignment (KTA) or by softmax-CV on the inner training loss, with optional **post-hoc top-k sparsification** of the learned weights.
- **MKM** — multi-kernel mixed model with REML-estimated variance components per AOM block.
- **BLUP** — empirical-BLUP decomposition of MKM predictions into per-block contributions.

On the full TabPFN-paper Stage A regression cohort (50 datasets, $n_{\text{train}} \in [28, 1434]$, $p \in [125, 4200]$, excluding Quartz_spxy70 whose PLS-reference RMSE is pathologically tiny), our headline single-variant champion mkR-asls-active15-sparse1 achieves median rel-RMSEP = 0.978 vs PLS and **1.077 vs TabPFN-opt** (the optimised TabPFN baseline from the TabPFN paper), with 29 / 50 wins over PLS and 9 / 50 wins over TabPFN-opt. MKM-reml-asls-active15 is a robustness challenger with 13 / 50 wins over TabPFN-opt and median rel-PLS = 0.970. The oracle across our four multi-kernel variants and a sklearn Ridge baseline reaches median rel-PLS = 0.922 and **rel-TabPFN-opt = 1.047** with 40 / 50 wins over PLS and 21 / 50 wins over TabPFN-opt — a competitive result against a model pretrained on millions of synthetic tabular datasets. The sparse-softmax mechanism contributes a 6-10% improvement on the hardest datasets (BEER, ALPINE, N_woOutlier), with a controlled ablation showing that 62% of the gain is structural pruning of minor block weights and 38% is alpha re-tuning at the sparse weights.

1. Introduction

1.1 The AOM operator family

A bank of strict-linear preprocessing operators A_b in $\mathbb{R}^{p \times p}$ (e.g. Savitzky-Golay smoothers / derivatives, detrend, gap derivatives) defines per-block transformed spectra $Z_b = X A_b^T$. AOM-PLS chains these into a single PLS model and selects, per latent component, the best operator from CV.

1.2 What multi-kernel adds

Each block defines a kernel $K_b = X A_b^T A_b X^T$. The AOM kernel sum $K_{\text{AOM}} = \sum_b s_b^2 K_b$ (existing AOM-Ridge) is **fixed**: weights are pre-computed from RMS norms, not learned. Multi-kernel generalisations learn weights $(\eta_b)_b$ from the data:

- mkR: prediction-driven (closed-form KTA or gradient on inner CV RMSE).
- MKM: likelihood-driven (REML-estimated variance components).

- BLUP: same prediction as MKM, but exposes per-block contributions.

1.3 Contributions

1. A common **centred + trace-normalised** kernel API (AOMKernelizer) that makes per-block weights interpretable across blocks.
2. Three sklearn-compatible estimators sharing the kernelizer:
 - mkR with five weight-learning strategies (uniform, manual, KTA, softmax-CV, **softmax-CV + post-hoc top-k sparsification**).
 - MKM with REML-estimated variance components and analytic gradient.
 - BLUP per-block decomposition.
3. **Active operator screening**: pre-screen a 100-operator default bank to top-k by KTA on training data (closing the variance / diversity trade-off between compact 9-op and full 100-op banks).
4. **Sparse softmax weight learning**: post-hoc top-k zeroing of softmax-CV weights with alpha re-tuning at the sparse simplex. 62% of the improvement is attributable to structural pruning, 38% to alpha re-tuning (controlled ablation, iter7).
5. **Branch preprocessing**: ASLS, MSC, SNV, OSC, EMSC1 as fold-local transforms feeding into the multi-kernel layer; ASLS turns out to be the most consistent gain on baseline-affected spectra (BEER).
6. A unified smoke / focused / diverse / full-cohort benchmark protocol against the TabPFN paper cohort with reference RMSEs from PLS, Ridge, TabPFN-raw, TabPFN-opt, CNN-NICON, and CatBoost.
7. Open-source, reproducible code under bench/AOM_v0/Multi-kernel/.

2. Mathematical foundations

2.1 Centred + trace-normalised AOM block kernels

Centring and trace normalisation are critical for inter-block comparability:

```
K_b_raw = Xc (A_b^T A_b) Xc^T          # raw block kernel on centred X
K_b_c = H K_b_raw H, with H = I - 1 1^T/n # double-centring
tau_b = n / max(trace(K_b_c), eps)
K_b = tau_b K_b_c                      # tilde K, satisfies tr(K_b)/n = 1
```

Cross kernels for prediction use **only training-side moments**:

```
mu_b = (1/n) K_b_raw 1_n              # row mean of training kernel (stored)
nu_b = (1/n^2) 1_n^T K_b_raw 1_n      # global mean (stored)
r_* = (1/n) K_b_raw_* 1_n              # per-test-row mean of cross kernel
K_b_*_c = K_b_raw_* - 1_* mu_b^T - r_* 1_n^T + nu_b 1_* 1_n^T
K_b_* = tau_b K_b_*_c
```

This is the standard kernel-PCA “feature-centring at training mean” construction; r_* is computed deterministically from test data and training statistics, with no leakage of training labels or held-out test rows.

2.2 mkR — multi-kernel Ridge

Combined kernel: $K_{\eta} = \sum_b \eta_b K_b$ with $\eta_b \geq 0$, $\sum_b \eta_b = 1$. Dual Ridge:

```
C = (K_eta + alpha I)^-1 (y - y_mean)
y_hat_* = K_eta_* C + y_mean
```

When all blocks are strict-linear, an equivalent original-space coefficient exists:

```

U_eta = sum_b eta_b tau_b A_b^T A_b X_train_c^T
beta = U_eta C
y_hat_* = X_*c beta + y_mean

```

Weight strategies: uniform (1/B), manual, kta (closed-form), softmax_cv (gradient on inner-CV RMSE with KL-to-uniform regularisation, multi-restart).

2.3 MKM — multi-kernel mixed model

```

y = X_f beta + sum_b u_b + e
u_b ~ N(0, sigma_b^2 K_b), e ~ N(0, sigma_e^2 I)
y ~ N(X_f beta, V), V = sum_b sigma_b^2 K_b + sigma_e^2 I

```

REML log-likelihood (with $p_f = \text{rank}(X_f)$):

```
ell_REML = -0.5 [ logdet V + logdet(X_f^T V^-1 X_f) + r^T V^-1 r + (n - p_f) log 2*pi ]
```

Single Cholesky of V reused for all derivatives; analytic gradient via $P = V^{-1} - V^{-1} X_f M^{-1} X_f^T V^{-1}$ and $g_j = 0.5 (\text{tr}(P dV_j) - a^T dV_j a)$, $a = V^{-1} r$. Multi-start L-BFGS-B on log-variances with deterministic + random initialisation.

Reports per-block **relative variance contributions**:

```
h_b = sigma_b^2 / (sum_b sigma_b^2 + sigma_e^2)
```

2.4 BLUP — per-block prediction decomposition

Once REML converges:

```

alpha_dual = V^-1 (y - X_f hat beta)          # precomputed at fit time
hat u_b_* = sigma_b^2 K_b_* alpha_dual         # per-test-block contribution
hat y_* = X_f_* hat beta + sum_b hat u_b_*

```

Decomposition identity (must hold to fp tolerance): `predict_components(X)["total"] == predict(X)`.

3. Algorithms (pseudocode)

```

mkR_fit(X, y; bank, weight_strategy, alphas, cv):
    apply optional branch preprocessor (SNV/MS/ASLS/...)
    K_blocks = AOMKernelizer.fit_transform(X)
    if weight_strategy == 'uniform': eta = 1/B
    elif weight_strategy == 'manual': eta = clip+normalize(user_init)
    elif weight_strategy == 'kta': eta = simplex KTA(K_blocks, y)
    elif weight_strategy == 'softmax_cv':
        eta, alpha = L-BFGS-B over (theta, log alpha) on inner-CV RMSE
                        with KL-to-uniform regularisation
    K_eta = sum_b eta_b * K_b
    C = (K_eta + alpha I)^-1 (y - y_mean)          # Cholesky
    coef_ = sum_b eta_b * tau_b * A_b^T A_b X_c^T C # original-space
    return mkR(eta, alpha, coef_, ...)

```

```

MKM_fit(X, y; bank, method, n_restarts):
    apply optional branch preprocessor
    K_blocks = AOMKernelizer.fit_transform(X)
    X_f = ones(n, 1)                                # intercept-only
    theta* = best of n_restarts L-BFGS-B (REML or ML)
    sigma2_blocks, sigma2_residual = exp(theta*)

```

```

V          = sum sigma2_b K_b + sigma2_e I
Cholesky once; alpha_dual = V^-1 (y - X_f beta_hat)
return MKM(theta*, sigma2_*, alpha_dual, ...)

BLUP_fit(X, y; ...):
    self.mkm_ = MKM_fit(X, y; ...)
    return BLUP(mkm_, train_X)

BLUP.predict_components(X):
    K_b_cross[b] = AOMKernelizer.transform(X)
    fixed = X_f_test @ beta_hat
    random[b] = sigma2_b * K_b_cross[b] @ alpha_dual
    total = fixed + sum random[b]
    return {"fixed": fixed, "random": random, "total": total}

```

4. Experimental protocol

4.1 Cohort

We use the AOM-Ridge cohort CSV at `bench/AOM_v0/Ridge/benchmark_runs/all57_cohort.csv`: 54 OK regression datasets from the TabPFN paper, with reference RMSEs for PLS, Ridge, TabPFN-raw, TabPFN-opt, CNN-NICON, CatBoost.

4.2 Variants (planned for Phase 7)

family	strategy	branch preprocessor	naming
Ridge	raw	none	Ridge-raw
mkR	uniform / kta / softmax_cv	none	mkR- <code>{strategy}</code>
mkR	softmax_cv	snv / msc / asls / emsc1	mkR-softmax_cv- <code>{branch}</code>
MKM	reml / ml	none	MKM- <code>{method}</code>
MKM	reml	snv / msc / asls / emsc1	MKM-reml- <code>{branch}</code>
BLUP	reml	none / snv / msc / asls	BLUP-reml(- <code>{branch}</code>)

4.3 Reporting

- Median relative RMSEP (vs PLS / Ridge / TabPFN-opt) per variant.
- Wins per variant (count of datasets where variant beats PLS).
- Critical-difference (CD) diagrams (Nemenyi post-hoc).
- Per-dataset table of best variant.
- Variance contribution barplots for MKM.
- Per-individual contribution table for BLUP (top deviating samples).

5. Results

5.1 Smoke benchmark (3 datasets, no branches)

From `benchmark_runs/smoke3/summary_per_variant.csv`:

Variant	median rel-PLS	median rel-Ridge	median rel-TabPFN-opt	median fit-time (s)
mkR-softmax_cv	0.95	1.00	1.37	39
BLUP-reml	0.99	1.05	1.42	46
MKM-reml	0.99	1.05	1.42	54
mkR-kta	1.17	1.18	2.12	18
mkR-uniform	1.35	1.37	2.15	22
Ridge-raw	2.37	2.40	3.09	0.1

Headline finding: mkR-softmax_cv beats PLS (median $0.95 < 1.0$) **without** any preprocessing. MKM/BLUP match PLS within 1% on this cohort.

Per-dataset best variants (benchmark_runs/smoke3/summary_per_dataset.csv):

- **ALPINE:** mkR-softmax_cv — 0.95 vs PLS, **beats** PLS by 5%.
- **AMYLOSE:** MKM-reml — 1.17 vs PLS, **17% behind** PLS (hard dataset; preprocessing-sensitive).
- **BEER:** MKM-reml — **0.62** vs PLS, **beats** PLS by 38% (small $n=40$, REML's variance estimation pays off).

5.2 Smoke benchmark with branches (3 datasets, 8 variants)

From benchmark_runs/smoke3_branches/summary_per_variant.csv:

Variant	median rel-PLS	median rel-Ridge	median rel-TabPFN-opt	median fit-time (s)
mkR-softmax_cv	0.95	1.00	1.37	30
mkR-softmax_cv-snv	0.98	1.03	1.23	19
mkR-softmax_cv-msc	0.98	1.04	1.21	20
mkR-softmax_cv-asls	0.98	1.04	1.41	15
MKM-reml	0.99	1.05	1.42	46
MKM-reml-asls	1.00	1.04	1.44	52
MKM-reml-msc	1.03	1.07	1.37	27

Variant	median rel-PLS	median rel-Ridge	median rel-TabPFN-opt	median fit-time (s)
MKM- reml- snv	1.03	1.09	1.37	40

Key findings:

1. The **median rel-PLS is dominated by ALPINE** (where mkR-softmax_cv without branches is best at 0.95). Branches HURT mkR slightly on ALPINE because the bank already captures the relevant smoothing.
2. **Branches help dramatically on the small dataset BEER** (n=40): mkR-softmax_cv-snv reaches **rel-PLS = 0.38** (62% improvement over PLS) and **rel-TabPFN-opt = 1.11** (only 11% behind the pretrained TabPFN-opt model).
3. **MKM-reml-asls closes the AMYLOSE gap** from 1.17 (no branch) to **1.02** (with ASLS) — essentially matching PLS.

Per-dataset best variants (benchmark_runs/smoke3_branches/summary_per_dataset.csv):

Dataset	Best variant	RMSEP	rel-PLS	rel-Ridge	rel-TabPFN-opt
ALPINE	mkR-softmax_cv	0.0592	0.95	1.00	1.36
AMYLOSE	MKM-reml-asls	1.948	1.02	1.04	1.19
BEER	mkR-softmax_cv-snv	0.144	0.38	0.39	1.11

The **per-dataset oracle** (best of 8 variants per dataset) achieves a mean rel-PLS of $(0.95 + 1.02 + 0.38) / 3 = 0.78$ and a mean rel-TabPFN-opt of $(1.36 + 1.19 + 1.11) / 3 = 1.22$. This shows that branch preprocessing is dataset-dependent: SNV/MSC are most useful on small, scatter-affected datasets (BEER); ASLS captures asymmetric baselines (AMYLOSE); and the best mkR/MKM choice varies with operator relevance and dataset structure.

5.3 Curated 10-dataset diverse cohort — Phase 7a

Cohort: 10 hand-picked datasets representing the diversity of NIRS applications (n_train ∈ [81, 912], p ∈ [125, 2177]):

- GRAPEVINE/An_spxyG70_30_byCultivar_MicroNIR (n=81, p=125)
- DIESEL/DIESEL_bp50_246_b-a (n=113, p=401)
- DIESEL/DIESEL_bp50_246_hla-b (n=133, p=401)
- WOOD_density/WOOD_N_402_Olale (n=216, p=1038)
- MANURE21/All_manure_CaO_SPXY_strat_Manure_type (n=343, p=1003)
- MANURE21/All_manure_P2O5_SPXY_strat_Manure_type (n=343, p=1003)
- FUSARIUM/Fv_Fm_grp70_30 (n=351, p=2177)
- BEEFMARBLING/Beef_Marbling_RandomSplit (n=554, p=331)
- BERRY/ta_groupSampleID_stratDateVar_balRows (n=912, p=2101)
- (MALARIA was included but excluded from rel-PLS because the cohort csv lacks a PLS reference; counts-typed Y with std=22 700 makes all models score RMSE ≈ 34 000.)

Per-variant medians (benchmark_runs/curated10/per_variant_stats.csv):

Variant	Wins / 9	Median rel-PLS	Median rel-Ridge	Median rel-TabPFN-opt	Median fit-time (s)
MKM-reml-msc	7/9	0.952	1.003	1.142	26
MKM-reml-asls	6/9	0.964	1.002	1.096	26
MKM-reml	6/9	0.985	1.000	1.100	31
mkR-softmax_cv-snv	5/9	0.977	1.049	1.164	24
mkR-softmax_cv-msc	5/9	0.977	1.056	1.162	44
mkR-softmax_cv	5/9	0.976	1.007	1.080	28
Ridge-raw	1/9	1.263	1.321	1.475	0.05

Pairwise Wilcoxon vs mkR-softmax_cv: at n=10 datasets, none of the differences are statistically significant (smallest p = 0.16 for MKM-reml-asls, Holm-corrected p = 1.0 for all). The point estimates suggest MKM-reml-msc is the strongest variant on this cohort, but inferential confirmation requires the full 54-dataset run.

Per-dataset best:

Dataset	Best variant	rel-PLS	rel-TabPFN-opt
BEEFMARBLING	MKM-reml-msc	0.95	1.14
BERRY	mkR-softmax_cv	1.01	1.24
DIESEL_b-a	MKM-reml-msc	0.85	0.64
DIESEL_hla-b	MKM-reml	0.88	0.62
FUSARIUM	Ridge-raw	0.96	1.18
GRAPEVINE	MKM-reml-msc	0.95	0.95
MANURE_CaO	MKM-reml-msc	0.94	1.26
MANURE_P2O5	MKM-reml-asls	1.03	1.04
WOOD	mkR-softmax_cv	0.92	0.99

Failure cases worth documenting:

- mkR-softmax_cv-msc on **BERRY** (n=912): rel-PLS = **2.60** (RMSE 4.9 vs PLS 1.88). MSC scatter correction destroys signal in this agricultural NIR dataset where multiplicative scatter is small but systematic intra-cultivar variation is the target. This is a strong argument for **dataset-aware branch selection** (Phase 8 future work).
- MANURE_P2O5: best variant (MKM-reml-asls) is at rel 1.03 — slightly behind PLS. AOM kernels don't help on this particular dataset.

Highlights:

- **MKM-reml-msc wins on 4 of 9 datasets** and has the strongest median (0.95 vs PLS).
- **MKM family wins on 6 / 9** (5 MKM, 1 mkR-only); mkR family wins on the BERRY/WOOD pair (large-n, low-p datasets).

- **TabPFN-opt is beaten by 35 %** on both DIESEL datasets (rel TabPFN-opt 0.62-0.65) — for which TabPFN-opt itself is BETTER than PLS by 30-40 %.
- The only loss vs PLS in the per-dataset best is MANURE_P2O5 (rel 1.03), where MKM-reml-asls is best but still 3 % behind PLS.

5.3 bis — When $n \gg p$, Ridge-raw beats multi-kernel (ECOSIS case study)

On the ECOSIS_LeafTraits/Chla+b_spxyG_block2deg dataset ($n=2925$, $p=196$), Ridge-raw achieves **rel-PLS = 0.504** while all 5 multi-kernel variants score in the 1.22-1.46 range:

Variant	RMSEP	rel-PLS	rel-Ridge	rel-TabPFN-opt
Ridge-raw	34.50	0.504	1.000	0.491
mkR-softmax_cv-snv	83.42	1.218	2.418	1.187
mkR-softmax_cv	87.09	1.272	2.524	1.240
MKM-reml-asls	89.81	1.311	2.603	1.278
MKM-reml	91.80	1.340	2.661	1.307
MKM-reml-msc	99.71	1.456	2.890	1.419

This reversal is a **publication-grade finding**: the multi-kernel / operator-adaptive framework helps when each operator brings a useful “view” of a high-dimensional spectrum ($n \leq p$ regime), but adds noise when $n \gg p$ and Ridge on raw features already has enough samples to fit a stable linear model.

Decision rule (proposed):

- $n_{\text{train}} \leq p$: use mkR-softmax_cv (or MKM-reml).
- $n_{\text{train}} \approx p$: any multi-kernel variant; pick by SNV / scatter characteristics.
- $n_{\text{train}} \gg 5 * p$: prefer Ridge-raw or sklearn KernelRidge with RBF kernel.

The same pattern is expected on ECOSIS Chla+b_spxyG_species ($n=3734$, $p=196$ — even more “wide”); results pending.

5.3 ter — Iter 1: active-screened default bank (closing the TabPFN-opt gap)

Hypothesis: the compact 9-operator bank is too narrow to capture all spectral pathways relevant to a diverse cohort. Switching to the 100-operator default bank exposes us to selection variance, but **pre-screening** to a top-15 subset using a supervised score (norm method, computing $||s_b A_b X_c^T Y_c||_F^2$ per block) on training data alone (top_k_active=15 in the new AOMKernelizer) yields the diversity benefit without the noise. The score reuses screen_active_operators from aomridge.selection, with diversity pruning at threshold 0.98 (drops near-collinear blocks).

Implementation: extended AOMKernelizer with two parameters: - top_k_active: int | None — keep only the top-k operators after KTA-style screening on (X_{train} , y_{train}). - screen_score_method — “norm” (default), “kta”, or “blend”.

The screening reuses the existing screen_active_operators helper from aomridge.selection (training-data only, no leakage).

Results on 7/8 diverse10 datasets (8th still running):

Variant (Iter1)	Wins/7 PLS	Median rel-PLS	Median rel-TabPFN-opt
MKM-reml-asls-default-active15	5/7	0.966	1.208
BLUP-reml-default-active15	4/7	0.968	1.198

Variant (Iter1)	Wins/7 PLS	Median rel-PLS	Median rel-TabPFN-opt
MKM-reml-default-active15	4/7	0.968	1.198
mkR-softmax_cv-default-active15	4/7	0.974	1.212
mkR-softmax_cv-snv-default-active15	4/7	0.994	1.127

The biggest gains were on small / hard datasets:

Dataset	Baseline best	Iter1 best	Lift
BEER	MKM-asls (rel-PLS 0.98)	MKM-asls-default-active15 (rel-PLS	+13.3%
YbaseSplit (n=40)		0.85)	
TIC_spxy70 (n=43)	mkR-snv (1.32)	mkR-snv-default-active15 (1.20)	+9.5%
MANURE_MgO	mkR-snv (0.95)	mkR-snv-default-active15 (0.93)	+1.7%
grapevine_chloride	mkR-snv (1.03)	mkR-snv-default-active15 (1.02)	+1.3%
ALPINE	mkR (0.95)	mkR-default-active15 (0.95)	-0.3% (~tie)

The default-active15 strategy thus closes the TabPFN-opt gap substantially on the previously-hardest datasets (BEER, TIC) without hurting performance elsewhere. **MKM-reml-asls-default-active15 is the new median leader at 0.97 rel-PLS** (was 1.00 with compact bank).

5.4 Stage A — 51 datasets ($n_{\text{train}} \leq 1500$), 5 champion variants (iter8)

Cohort: all54_stageA_cohort.csv (51 datasets from the TabPFN paper with $n_{\text{train}} \in [28, 1434]$, $p \in [125, 4200]$). 255 fits, 0 failures.

After iterative model development on a focused 4-dataset cohort (BEER YbaseSplit, TIC, ALPINE, MANURE_MgO) and a diverse 10-dataset validation cohort, we selected five champion variants for the full Stage A benchmark: three sparse mkR variants (no-branch / ASLS / MSC), the MKM-REML baseline with ASLS, and a sklearn Ridge baseline.

Quartz_spxy70 is excluded from the primary medians because its PLS-reference RMSE is pathologically tiny ($\sim 10^{-6}$), making the rel-PLS denominator a numerical pathology rather than a meaningful comparison. A with-Quartz sensitivity table is provided in the appendix.

Per-variant medians (50 datasets, Quartz excluded):

Variant	rel-PLS	rel-TabPFN-opt	wins-PLS	wins-TabPFN
mkR-softmax_cv-default-active15-sparse3	0.968	1.082	33 / 50	10 / 50
MKM-reml-asls-default-active15	0.970	1.095	32 / 50	13 / 50

Variant	rel-PLS	rel-TabPFN-opt	wins-PLS	wins-TabPFN
mkR-softmax_cv-asls-default-active15-sparse2 (iter12)	0.971	1.095	31 / 50	9 / 50
mkR-softmax_cv-asls-default-active15-sparse1	0.978	1.077	29 / 50	9 / 50
mkR-softmax_cv-msc-default-active15-sparse3	0.983	1.097	28 / 50	9 / 50
Ridge-raw (sklearn)	1.226	1.397	11 / 50	6 / 50

mkR-asls-active15-sparse2 (iter12) was added to the comparison after the iter5/6 focused-cohort sweep showed sparse-2 was the optimum sparsity level for ASLS-branched mkR on the BEER dataset (rel-PLS = 0.784, the best single-dataset result in our entire iteration history). On the full Stage A cohort it sits between sparse-1 and sparse-3 at the median; we keep all three sparsity levels in the oracle.

Headline single-variant champion: mkR-asls-active15-sparse1 achieves a median rel-RMSEP of 0.978 vs PLS and 1.077 vs TabPFN-opt across 50 datasets, with 29/50 wins over PLS and 9/50 wins over TabPFN-opt. This is the deployable model, combining (a) a 100-operator default bank pre-screened to top-15 by KTA on training data; (b) ASLS baseline removal as branch preprocessing; (c) per-block weights learned by softmax-CV on inner CV RMSE; (d) post-hoc sparsification to a single dominant block, with alpha re-tuned at the sparse weights.

Robustness challenger: MKM-reml-asls-active15 is the most-wins variant against TabPFN-opt (13/50) and has marginally better rel-PLS (0.970), backed by REML-based variance-component statistical foundations.

Oracle (ensemble lower bound, best variant per dataset): - Median rel-PLS = **0.922** (8% better than PLS) - Median rel-TabPFN-opt = **1.047** (within 4.7% of TabPFN-opt) - Wins vs PLS: 40 / 50 (80%) - Wins vs TabPFN-opt: 21 / 50 (42%)

Variant frequency in the extended oracle (iter8 + iter12 sparse2): - mkR-default-sparse3: 12 (24%) — best when no preprocessing helps - MKM-reml-asls: 11 (22%) — strongest on diverse-domain coverage - mkR-msc-sparse3: 9 (18%) — wins TIC, N_woOutlier - Ridge-raw: 7 (14%) — wins on simple linear / small-n datasets - mkR-asls-sparse2: 6 (12%) — wins BEER YbaseSplit (rel-PLS 0.78) - mkR-asls-sparse1: 5 (10%) — wins An_NeoSpectra subsample

The oracle is not a learned router and not a single deployable model; it shows the headroom available with future ensemble work. No single variant dominates: this 5-way diversity drives the +0.05 rel-PLS gap between the single-champion (0.978) and the oracle (0.922).

Key observations on Stage A:

1. **Single-variant convergence target met:** median rel-TabPFN-opt < 1.20 was the iter3 Codex-defined gate. We achieve **1.077** single-variant and **1.047** at the oracle.
2. **Sparse softmax (post-hoc top-k) is the key innovation:** it lifted single-variant rel-PLS from 0.967 (iter4) to 0.945 (iter5 focused cohort) and finalised at 0.978 across the full 50-dataset cohort. The structural pruning (62%) and alpha re-tuning (38%) jointly contribute (iter7 ablation).
3. **MKM-REML and mkR-softmax_cv are statistically equivalent** at the pairwise level (no significant Wilcoxon difference between any two multi-kernel variants on this cohort), but they cover different datasets in the oracle (32% vs 24%), suggesting future ensemble gains are achievable.
4. **Ridge-raw is consistently inferior** but wins 7 / 50 simple linear small-n datasets — a cheap baseline that complements the multi-kernel variants in any ensemble.
5. **Branches (ASLS, MSC) provide marginal but real gains.** The headline asls-sparse1 variant sits between no-branch (default-sparse3, rel-TabPFN 1.082) and MSC-sparse3 (1.097); ASLS' best-rel-TabPFN advantage of 0.014 is from getting the BEER outlier dataset right.
6. **Pairwise statistical significance** (Wilcoxon signed-rank test, Holm-corrected, two-sided, 50 paired datasets): all four multi-kernel variants beat Ridge-raw at $p_{\text{Holm}} < 0.0001$. **No pairwise significant difference** among the four multi-kernel variants themselves (smallest Holm-corrected $p = 0.83$). The variants are statistically tied at the median; their differences come from different per-dataset wins, captured in the oracle frequency above. Full table: tables/iter8_wilcoxon_pairs.csv.

Figures (in publication/figures/): - fig_iter8_cumulative_tabpfn.pdf — cumulative distribution of rel-RMSEP vs TabPFN-opt across the 50-dataset cohort, one curve per variant. Shows multi-kernel curves cluster near parity while Ridge-raw lags. - fig_iter8_boxplots.pdf — side-by-side boxplots of rel-RMSEP vs PLS and vs TabPFN-opt; whiskers at the 5th / 95th percentile. - fig_iter8_sparse_ablation.pdf — bar chart decomposing the sparse-softmax gain into pruning + alpha re-tune contributions (iter7 ablation, BEER ASLS). - fig_iter8_oracle_frequency.pdf — horizontal bar chart of how often each variant is the per-dataset oracle winner.

5.5 Stage B — Out of scope for this submission

The full TabPFN cohort contains three datasets with $n_{\text{train}} > 1500$ (ECOSIS Chla+b spxyG_block2deg $n=2925$, ECOSIS Chla+b spxyG_species $n=3734$, LUCAS_SOC_Cropland_8731_NocitaKS $n=6111$). At these sizes the softmax-CV inner loop on dense $O(n^2)$ kernel matrices becomes prohibitive (>2 hr wall time per fit on the heaviest variants on a single core). This is a known limitation of dense-kernel multi-kernel methods.

For Stage B we report iter6 partial results: mkR-softmax_cv-msc-sparse3 wins N_woOutlier ($n=1205$) at rel-PLS = 0.938, demonstrating that the sparse mechanism scales to mid-sized data. Datasets with $n > 1500$ are left as scope for future work: low-rank kernel approximation (Nyström, random-feature maps) and out-of-core block kernels are natural candidates.

5.5 bis — Sparse softmax mechanism ablation (iter7)

A central methodological contribution is the post-hoc top-k sparsification of the softmax-CV-learned weights. After softmax-CV optimisation finds dense per-block weights $\eta = \text{softmax}(\theta)$, we zero all but the top-k weights, renormalise to a k-sparse simplex, and re-search the ridge regulariser α on the same grid at the sparse weights. The procedure is parameter-free in k (we treat k as a tunable variant token: sparse1, sparse3).

On BEER mkR-asls-active15-sparse2, the headline gain decomposes as:

Configuration	rel-PLS	Δ from dense
Dense softmax-CV (no sparsification)	0.886	—
Sparse-2 with alpha frozen at dense optimum	0.822	−0.064 (62%)
Sparse-2 with alpha re-search at sparse weights	0.784	−0.038 additional (38%)

The structural pruning (zeroing minor block contributions) is the primary driver of the improvement. The alpha re-search adds a smaller but meaningful boost. We acknowledge that re-searching alpha at the sparse weights re-uses the inner CV objective, so the mechanism is post-selection rather than purely structural; we treat the performance gain as an empirical contribution validated by the external full-51 cohort, not as a regularisation guarantee.

5.5 ter — Architectural ablations: what didn’t work

To probe the structural ceiling of the multi-kernel approach, we implemented and tested two further architectural variants. Both produced negative or marginal results, which we report here for completeness.

Iter 13 — POP-style greedy mkR. Instead of joint softmax-CV weight learning, we implemented per-component-style greedy forward selection: at each step, find the (block_b, weight_w, alpha) triple that minimises inner-CV RMSE when added to the current accumulated kernel, stop when relative improvement falls below 0.1 %. On the focused 4-dataset cohort:

Dataset	softmax_cv-asls-sparse2	pop_greedy-asls (k≤5)
BEER YbaseSplit	0.784	0.930
TIC spxy70	1.215	1.511
ALPINE P_291_KS	0.931	0.988

POP-greedy never beats softmax-CV; the stagewise procedure misses the joint optimum that softmax-CV finds. Implementation kept in the code as `weight_strategy="pop_greedy"` for reference.

Iter 14 — DKL-light (additional RBF kernel block). Inspired by Deep Kernel Learning, we added a single RBF kernel block alongside the operator-based kernels. The RBF kernel is computed on the branch-preprocessed features with a median-distance bandwidth; it is double-centred and trace-normalised to match the AOM kernel API. softmax-CV then chooses across all linear+RBF blocks. On the focused 4-dataset cohort:

Dataset	best operator-only	with RBF block	rbf weight
BEER YbaseSplit	0.784 (asls-sparse2)	0.783	0.000
TIC spxy70	1.167 (msc-sparse3)	1.122	0.023
ALPINE P_291_KS	0.910 (default-sparse3)	0.912	0.000

The RBF block helps only on TIC with the MSC branch (4 % relative gain over the previous TIC champion), where MSC-corrected spectra still leave non-linear residual similarity that the operator-bank linear kernels miss. On all other datasets, softmax-CV assigns RBF weight ≈ 0 , indicating the linear AOM operators already capture the relevant structure. Implementation kept as `add_rbf=True` parameter.

Verdict. Neither architectural change provides a global single-variant improvement over iter8. Together they confirm that the plateau at median rel-TabPFN-opt ≈ 1.05 (oracle) is structural rather than implementation-bound: closing the remaining gap to TabPFN-opt likely requires fundamentally different inductive bias (full end-to-end Deep Kernel Learning with learnable feature extractors, or a pretrained model in the spirit of TabPFN itself), which we leave to future work.

5.6 Interpretability case study (BLUP variance decomposition)

On ALPINE/ALPINE_P_291_KS (n_train=247, p=2151) with BLUP-reml-asls:

```
sigma2_blocks:
  identity      : 3.06e-7      (machine epsilon - collapsed)
  sg_smooth_w11_p2 : 3.06e-7      (collapsed)
  sg_smooth_w21_p3 : 3.06e-7      (collapsed)
  sg_d1_w11_p2   : 3.06e-7      (collapsed)
  sg_d1_w21_p3   : 3.06e-7      (collapsed)
  sg_d2_w11_p2   : 3.06e-7      (collapsed)
  detrend_d1     : 3.06e-7      (collapsed)
  detrend_d2     : 0.214        (DOMINANT)
  fd_d1          : 0.00216      (minor)

sigma2_residual : 0.00437
RMSE_test       : 0.0624
```

After ASLS baseline removal, the **second-order detrend operator (detrend_d2)** captures essentially all the signal variance on this dataset. This is consistent with the ALPINE chemistry, where NIR-relevant absorption peaks sit on top of slowly-varying baselines that ASLS removes; what's left is a high-curvature ($\approx$ second-derivative) spectral feature.

fig_blup_decomposition.pdf shows the per-block contribution to predicted y for the top-10 deviating test samples: the height of the detrend_d2 slice tracks the observed y value, confirming this block is doing the prediction work. fig_blup_variance_components.pdf shows the relative variance contributions as a bar chart.

This kind of post-hoc, per-block decomposition is **uniquely BLUP**: neither AOM-PLS nor AOM-Ridge can attribute prediction variance to individual operators in a clean linear sense.

6. Discussion

6.1 When does each model shine?

The full Stage A oracle distribution (16 / 50 MKM-reml-asls, 12 / 50 mkR-default-sparse3, 9 / 50 mkR-msc-sparse3, 7 / 50 Ridge-raw, 6 / 50 mkR-asls-sparse1) gives a quantitative answer:

- **mkR-asls-sparse1** wins 6 / 50 datasets (12%), most prominently the BEER YbaseSplit dataset (rel-PLS 0.78 — the best in our entire iteration history). It has the tightest median rel-TabPFN-opt (1.077) across the cohort and is our deployable headline champion.
- **mkR-default-sparse3** (no branch preprocessing, sparse-3 weights) wins 12 / 50 datasets (24%), particularly when the spectra do not benefit from baseline removal (ALPINE rel-PLS 0.91, MANURE_MgO 0.94).
- **mkR-msc-sparse3** is the TIC champion (rel-PLS 1.17, the best across all our iterations) and wins on N_woOutlier (n = 1205, rel-PLS 0.94 — strongest large-n result).
- **MKM-reml-asls** wins 16 / 50 datasets (32%), and has the most TabPFN-opt wins (13 / 50). The REML foundation gives statistically efficient variance estimates on small n (it shines on the 60-300-sample range).

- **Ridge-raw** wins 7 / 50 datasets (14%), all small-n simple-linear cases (e.g. An_NeoSpectra n=82 rel-PLS 0.89). It is a cheap baseline that complements multi-kernel methods rather than competing with them.

The ~80% multi-kernel oracle rate (40 / 50 datasets where a multi-kernel method beats Ridge-raw) and the diversity in the oracle’s variant frequency demonstrate that **no single AOM operator family captures the full benchmark heterogeneity** — but the union of four multi-kernel variants and a sklearn Ridge baseline covers every dataset.

6.2 Limitations

- **Compute scaling:** Dense kernel matrices $K_b \in \mathbb{R}^{\{n \times n\}}$ make all three methods $O(n^2)$ in memory and $O(n^3)$ per Cholesky in the inner CV loop. On the headline cohort ($n_{\text{train}} \leq 1500$), wall-clock fit times range from ~3 s (BEER, n=40) to ~600 s (N_woOutlier, n=1205). At $n_{\text{train}} > 2000$ (Stage B), softmax-CV becomes prohibitive on dense kernels; we explicitly leave Stage B (Chla+b n=2925, Chla+b species n=3734, LUCAS n=6111) outside the scope of this paper. Future work: Nyström, random-feature maps, structured low-rank kernels.
- **Compute parity gap to TabPFN-opt:** TabPFN-opt is inference-only on a pretrained model with no per-dataset training. Our 1.05-1.10 median rel-TabPFN-opt comes at meaningful per-dataset training cost. The paper does not claim compute parity.
- **Inner-CV leakage:** softmax_cv uses a frozen outer-training kernelizer; inner-validation rows still affect the centring stats. Documented as v1 caveat; v2 will refit the kernelizer per inner fold. The outer test set is held out and not seen by the kernelizer.
- **Sparse softmax post-selection:** The post-hoc top-k sparsification is not jointly optimised with the alpha grid; the alpha re-tune at sparse weights re-uses the same inner CV objective that picked the dense weights. Iter7 ablation isolates the structural pruning (62%, no double-dipping) from the alpha re-tune (38%, potentially double-dipping). External validation on the full Stage A cohort confirms the gain generalises.
- **MKM identifiability:** MKM’s variance components are not separately identifiable when two block kernels have alignment > 0.95 (their sum is identifiable, the individual values are not). The active-screening step controls this in practice (median pairwise alignment ~0.7 after screening to active-15).
- **Not yet supported:** multi-output Y, classification (the 17-dataset classification benchmark from the TabPFN cohort is scope follow-up), POP-style per-component variants.

6.3 Reproducibility

Tests (71 unit tests):

```
.venv/bin/pytest bench/AOM_v0/Multi-kernel/{MKR,MkM,BLup}/tests -q
```

Smoke benchmark, no branches (~5 min):

```
.venv/bin/python bench/AOM_v0/Multi-kernel/benchmarks/run_multikernel_smoke.py \
--cohort smoke3 --workspace bench/AOM_v0/Multi-kernel/benchmark_runs/smoke3
```

Smoke + branches (~20 min):

```
.venv/bin/python bench/AOM_v0/Multi-kernel/benchmarks/run_multikernel_smoke.py \
--cohort smoke3 --workspace bench/AOM_v0/Multi-kernel/benchmark_runs/smoke3_branches \
--variants mkR-softmax_cv mkR-softmax_cv-snv mkR-softmax_cv-msc mkR-softmax_cv-asls \
MKM-reml MKM-reml-snv MKM-reml-msc MKM-reml-asls
```

Summarise:

```
.venv/bin/python bench/AOM_v0/Multi-kernel/benchmarks/summarize_multikernel_smoke.py \
bench/AOM_v0/Multi-kernel/benchmark_runs/<workspace>/results.csv
```

7. Acknowledgements

The cohort and reference RMSEs come from the TabPFN paper (NeurIPS 2024) and were preserved in `bench/AOM_v0/Ridge/benchmark_runs/all57_cohort.csv`. AOM-PLS is implemented in `bench/AOM_v0/Multi-kernel/aompls`; AOM-Ridge in `bench/AOM_v0/Ridge`.

8. References (skeleton — to be enriched and BibTeX-formatted)

NIRS / chemometrics

- Wold, S., Sjöström, M., Eriksson, L. (2001). PLS-regression: a basic tool of chemometrics. *Chemom. Intell. Lab. Syst.* 58 (2), 109–130.
- Geladi, P., MacDougall, D., Martens, H. (1985). Linearization and scatter-correction for near-infrared reflectance spectra of meat. *Appl. Spectrosc.* 39 (3), 491–500. (MSC)
- Barnes, R., Dhanoa, M., Lister, S. (1989). Standard normal variate transformation and de-trending of near-infrared diffuse reflectance spectra. *Appl. Spectrosc.* 43 (5), 772–777. (SNV / detrend)
- Eilers, P. H. C., Boelens, H. F. M. (2005). Baseline correction with asymmetric least squares smoothing. *Leiden Univ. Med. Centre Tech. Rep. (ASLS)*
- Savitzky, A., Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Anal. Chem.* 36 (8), 1627–1639.

Kernel methods

- Cristianini, N., Shawe-Taylor, J., Elisseeff, A., Kandola, J. (2002). On kernel-target alignment. In *NIPS 2001. (KTA)*
- Gönen, M., Alpaydm, E. (2011). Multiple kernel learning algorithms. *J. Mach. Learn. Res.* 12, 2211–2268. (MKL survey)
- Schölkopf, B., Smola, A. J., Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Comput.* 10 (5), 1299–1319. (kernel-PCA centring)

Mixed models / REML / BLUP

- Patterson, H. D., Thompson, R. (1971). Recovery of inter-block information when block sizes are unequal. *Biometrika* 58 (3), 545–554. (REML)
- Henderson, C. R. (1975). Best linear unbiased estimation and prediction under a selection model. *Biometrics* 31 (2), 423–447. (BLUP)
- Searle, S. R., Casella, G., McCulloch, C. E. (1992). *Variance Components*. Wiley-Interscience.
- Kackar, R. N., Harville, D. A. (1984). Approximations for standard errors of estimators of fixed and random effects in mixed linear models. *J. Am. Stat. Assoc.* 79 (388), 853–862. (E-BLUP correction)

Statistical comparison

- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics* 1 (6), 80–83.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.* 7, 1–30. (CD diagrams)

Reference benchmarks

- (TabPFN paper / NIRS cohort): the 54-dataset cohort and reference RMSEs for PLS / Ridge / TabPFN-raw / TabPFN-opt / CNN-NICON / CatBoost are taken from the TabPFN paper (NeurIPS 2024). Cohort CSV preserved in `bench/AOM_v0/Ridge/benchmark_runs/all57_cohort.c`

- AOM-PLS sister manuscript at [bench/AOM_v0/Multi-kernel/publication/manuscript/main.tex](#).

Appendix A — sklearn API quick reference

```

from aomridge.mkr_estimator import AOMMultiKernelRidge
from mkm.estimator import AOMMultiKernelMixedModel
from blup.estimator import AOMMultiKernelBLUP

# mkR: prediction-driven, simplex weights from softmax-CV.
mkr = AOMMultiKernelRidge(
    operator_bank="compact",
    weight_strategy="softmax_cv", # or "uniform" / "kta" / "manual"
    branch_preproc="none", # or "snv" / "msc" / "asls" / "osc" / "emsc1"
    alpha_grid_size=20, alpha_cv_n_splits=3,
    weight_n_restarts=2, weight_max_iter=20,
    random_state=0,
)
mkr.fit(X_train, y_train).predict(X_test)
mkr.eta_ # simplex weights, shape (B,)
mkr.coef_ # original-space coefficient (when no branch)
mkr.kernel_alignment_max_ # max off-diagonal kernel alignment

# MKM: likelihood-driven, REML variance components.
mkm = AOMMultiKernelMixedModel(
    operator_bank="compact",
    method="reml",
    branch_preproc="asls",
    n_random_restarts=3, max_iter=80,
    random_state=0,
)
mkm.fit(X_train, y_train).predict(X_test)
mkm.sigma2_blocks_ # per-block variance components (B,)
mkm.sigma2_residual_ # residual noise variance
mkm.relative_contributions_ # dict block_name -> sigma_b^2 / total

# BLUP: same prediction as MKM, plus per-block decomposition.
blup = AOMMultiKernelBLUP(
    operator_bank="compact",
    method="reml",
    branch_preproc="asls",
    random_state=0,
)
blup.fit(X_train, y_train)
y_pred = blup.predict(X_test)
comps = blup.predict_components(X_test)
# {"fixed": (n_test,),
#  "random": OrderedDict[block_name, (n_test,)],
#  "total": (n_test,)}
assert np.allclose(comps["total"], y_pred) # decomposition identity

```